MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini-Project Report

Project Title

GPU-Based Parking Slot Detection using CUDA and Edge Density Analysis

SUBMITTED BY

Shreehari V Bhat (230905034)

Sathwik Padmanabha Bhat (230905012)

Abhishek Jayanth Holla (230905042)

Under the Guidance of:

Ashutosh Holla B

Assistant Professor

School of Computer Engineering

MIT Manipal

School of Computer Science and Engineering

Manipal Institute of Technology, Manipal, Karnataka – 576104

ABSTRACT

This report presents the design, implementation, and performance evaluation of a GPU-based parking slot occupancy detection system using NVIDIA CUDA and edge density analysis. Automated parking detection has become a critical requirement in modern urban infrastructure, where manual monitoring is both costly and error-prone. The proposed system addresses this challenge through a fully parallel, training-free image processing pipeline implemented in CUDA C.

The system processes overhead parking lot images through three sequential CUDA kernels: (1) an rgb_to_gray kernel that converts the input RGB image to a single-channel grayscale representation using the ITU-R BT.601 weighted formula, (2) a sobel_kernel that applies the Sobel operator using a Manhattan approximation (|Gx| + |Gy|) and thresholds the result at a magnitude of 80 to generate a binary edge map, and (3) a classify_roi_kernel that utilizes shared memory and atomicAdd() operations to compute the edge pixel density within each predefined bounding box Region of Interest (ROI).

Classification is performed by comparing the computed edge density against a threshold of 0.35, slots with density exceeding this threshold are marked as Occupied (red border), while the remainder are marked as Vacant (green border). The output is saved as final_result.png using the stb_image_write library, requiring no external computer vision framework such as OpenCV. Performance benchmarking demonstrates that for large images (2400×1440 pixels), the CUDA GPU implementation achieves approximately 8.35x speedup over the equivalent sequential CPU implementation.

CHAPTER 1: INTRODUCTION

1.1 General Introduction to the Topic

Parking management is a critical and increasingly complex problem in modern urban environments. As vehicular density continues to rise in metropolitan areas, the efficient utilization of available parking infrastructure directly impacts traffic flow, fuel consumption, and the quality of urban life. Traditional parking management systems rely on manual monitoring by security personnel, which is inherently inefficient, costly, and highly prone to human error. Drivers often spend considerable time circling parking lots searching for available spaces, contributing to traffic congestion and unnecessary fuel expenditure.

Automated parking detection systems aim to identify available parking slots in real time using sensors, cameras, or computational approaches. Vision-based methods, in particular, have gained significant attention due to the decreasing cost and increasing resolution of overhead cameras. These methods can be broadly categorized into classical image processing approaches, which rely on feature extraction and rule-based classification, and deep learning approaches, which train convolutional neural networks on large annotated datasets to achieve high detection accuracy.

Classical image processing methods are computationally lightweight but may suffer from reduced accuracy under varying lighting or environmental conditions. Deep learning methods, while more accurate, require substantial labeled training data and significant computational resources for inference. A critical gap exists in the literature: the application of GPU-accelerated parallel processing to simple, training-free edge-density-based classification approaches. This project bridges that gap by implementing a complete CUDA-based parking detection pipeline that is fast, lightweight, and free of machine learning dependencies.

1.2 Organization of the Report

The report is organized as follows:

- Chapter 2 defines the problem context and identifies the specific research gaps addressed by this work.
- Chapter 3 lists the primary technical objectives of the project.
- Chapter 4 provides background on CUDA parallel computing, edge detection, and image processing fundamentals.
- Chapter 5 describes the proposed methodology and the complete system pipeline.
- Chapter 6 presents the implementation details, including all CUDA kernels and CPU baseline code.
- Chapter 7 discusses the experimental results with visual output examples.

- Chapter 8 presents a comparative analysis of CPU versus GPU performance across multiple image resolutions.
- Chapter 9 concludes the report and outlines directions for future work.
- Chapter 10 lists all references cited in this report.

1.3 Area of Computer Science

This project falls within the domains of Parallel Computing, GPU Programming, and Computer Vision. Specifically, it addresses the application of NVIDIA CUDA's SIMT (Single Instruction, Multiple Thread) execution model to image processing tasks. The project demonstrates how classical Sobel edge detection, a well-established technique from the field of computer vision, can be significantly accelerated through GPU parallelism to achieve real-time or near-real-time performance on high-resolution parking imagery.

1.4 Hardware and Software Requirements

Hardware: NVIDIA GPU with CUDA Compute Capability 3.0 or higher, multi-core CPU.

Software: CUDA Toolkit 11.0+, NVCC compiler, GCC/G++ (Linux) or MSVC (Windows), stb_image.h and stb_image_write.h (single-header image I/O libraries).

Operating System: Linux (Ubuntu 20.04+) or Windows 10/11 with CUDA SDK installed.

CHAPTER 2: PROBLEM DEFINITION

Urban parking management faces a fundamental scalability and efficiency challenge. As cities grow and the number of registered vehicles increases, the demand for reliable, real-time parking availability information has never been greater. The following specific problems motivate this project:

- Many existing automated detection methods rely on deep learning models that require large, carefully annotated datasets for training. Acquiring and maintaining such datasets is resource-intensive and limits the generalizability of these models to new parking environments without retraining.
- Sensor-based systems (infrared, ultrasonic, or inductive loop sensors) require extensive physical installation and ongoing hardware maintenance, significantly increasing deployment cost per parking slot.
- Classical image processing methods for parking detection are often implemented on CPU hardware in a sequential manner, making them inadequate for real-time analysis of high-resolution surveillance images without dedicated hardware.
- There is a notable lack of GPU-based parallel implementations for simple, lightweight parking detection approaches. Most GPU-accelerated solutions in the literature focus exclusively on deep learning inference acceleration, rather than classical edge-based methods.
- Parking detection systems based on fixed thresholds are sensitive to changes in lighting conditions, weather, and camera angle, limiting their robustness in real-world deployments.

This project specifically addresses the gap identified in the third and fourth points: the absence of a GPU-parallelized, training-free, edge-density-based parking slot classifier that can process high-resolution images with significantly reduced latency compared to CPU sequential implementations.

CHAPTER 3: OBJECTIVES

The primary and technical objectives of this project are as follows:

1. To implement a complete parking slot occupancy detection pipeline using CUDA C without any external computer vision library dependencies.
2. To accelerate the grayscale conversion, Sobel edge detection, and ROI-based classification steps through GPU parallelism using the NVIDIA CUDA programming model.
3. To implement the Sobel edge detection kernel using a Manhattan magnitude approximation (|Gx| + |Gy|) with a binary threshold of 80, producing a memory-efficient binary edge map.
4. To define parking slot Regions of Interest (ROIs) using BBox{minx, miny, maxx, maxy} bounding box coordinates and process each slot independently in a separate CUDA block.
5. To apply CUDA shared memory (via the __shared__ keyword) within the classify_roi_kernel to minimize global memory access overhead during edge pixel counting.
6. To use atomicAdd() for race-condition-free parallel accumulation of edge pixel counts across 256 threads within each block.
7. To classify each slot as Occupied or Vacant based on a density threshold of 0.35, and render the result by drawing red or green pixel-level borders on the output image.
8. To save the annotated output as final_result.png using stb_image_write without OpenCV dependency.
9. To implement an equivalent CPU baseline for all three stages and quantitatively compare execution time against the GPU implementation across multiple image resolutions.
10. To analyze the scaling behavior of the GPU speedup as image resolution increases.

CHAPTER 4: BACKGROUND

4.1 CUDA Parallel Computing

CUDA (Compute Unified Device Architecture) is a parallel computing platform and programming model developed by NVIDIA that enables general-purpose computation on Graphics Processing Units (GPUs). Unlike CPUs, which are optimized for low-latency serial execution of complex instruction sequences, GPUs are designed with thousands of smaller, highly efficient cores optimized for high-throughput data-parallel workloads. CUDA exposes this parallelism through a hierarchical execution model: threads are grouped into blocks, and blocks are grouped into a grid. All threads within a block execute the same kernel function but operate on different data elements, following the Single Instruction, Multiple Thread (SIMT) paradigm.

In the context of image processing, each pixel of an image can be processed independently by a dedicated CUDA thread, making image kernels (such as grayscale conversion and Sobel filtering) naturally data-parallel. The two-dimensional thread block configuration dim3(16, 16), generating 256 threads per block, is well-suited to the two-dimensional structure of image data, with each thread mapped to a unique (x, y) pixel coordinate through blockIdx and threadIdx calculations.

4.2 CUDA Memory Hierarchy

CUDA provides several memory spaces that differ in size, access latency, and scope. Global memory is the largest but slowest memory space, accessible by all threads across all blocks. Shared memory is an on-chip, low-latency memory space shared among all threads within a single block. Using shared memory to aggregate per-block results (such as edge pixel counts) before writing to global memory significantly reduces memory access latency. The __shared__ keyword in CUDA C declares a variable in shared memory. Proper use of __syncthreads() barriers ensures that all threads in a block complete their write operations before any thread reads from shared memory, preventing data hazards.

4.3 Grayscale Conversion

Grayscale conversion reduces a three-channel RGB image to a single-channel luminance representation. This simplification is a necessary preprocessing step for gradient-based edge detection, which operates on intensity values rather than colour channels. The standard ITU-R BT.601 formula is used:

$$\mathbf{Gray = 0.299 \times R + 0.587 \times G + 0.114 \times B}$$

The weighting coefficients reflect the relative sensitivity of the human visual system to the three primary colours, with green contributing the most to perceived luminance, followed by red, and blue contributing the least.

4.4 Sobel Edge Detection

The Sobel operator is a discrete differentiation operator that computes an approximation of the gradient of the image intensity function at each pixel. It applies two 3×3 convolution kernels, one for horizontal gradient (Gx) and one for vertical gradient (Gy), to detect edges in both orientations. In this implementation, the gradient magnitude is computed using the Manhattan (L1) norm rather than the Euclidean norm, as it avoids the computationally expensive square root operation:

```
mag = |Gx| + |Gy|   →   edge[y×w + x] = (mag > 80) ? 255 : 0
```

Pixels where the gradient magnitude exceeds the threshold of 80 are classified as edge pixels (assigned value 255), while all other pixels are classified as non-edge (assigned value 0), producing a binary edge map.

4.5 Edge Density as an Occupancy Feature

The fundamental observation motivating this system is that an occupied parking slot contains a vehicle, which introduces numerous strong edges, from the car body outline, windows, tyres, and internal structural lines, that do not appear in an empty slot. An empty slot contains only the faint boundary markings of the parking space on the asphalt surface, producing very few edge pixels. Edge density, defined as the ratio of edge pixels to total pixels within a slot's bounding box, therefore provides a reliable discriminating feature between occupied and vacant states.

CHAPTER 5: PROPOSED METHODOLOGY

The proposed system processes a parking lot image through a five-stage pipeline, as illustrated in Figure 5.1 below. Each computational stage (grayscale conversion, Sobel edge detection, and ROI classification) is implemented as a CUDA kernel to exploit the inherent data parallelism of pixel-level image operations. The stages are as follows:

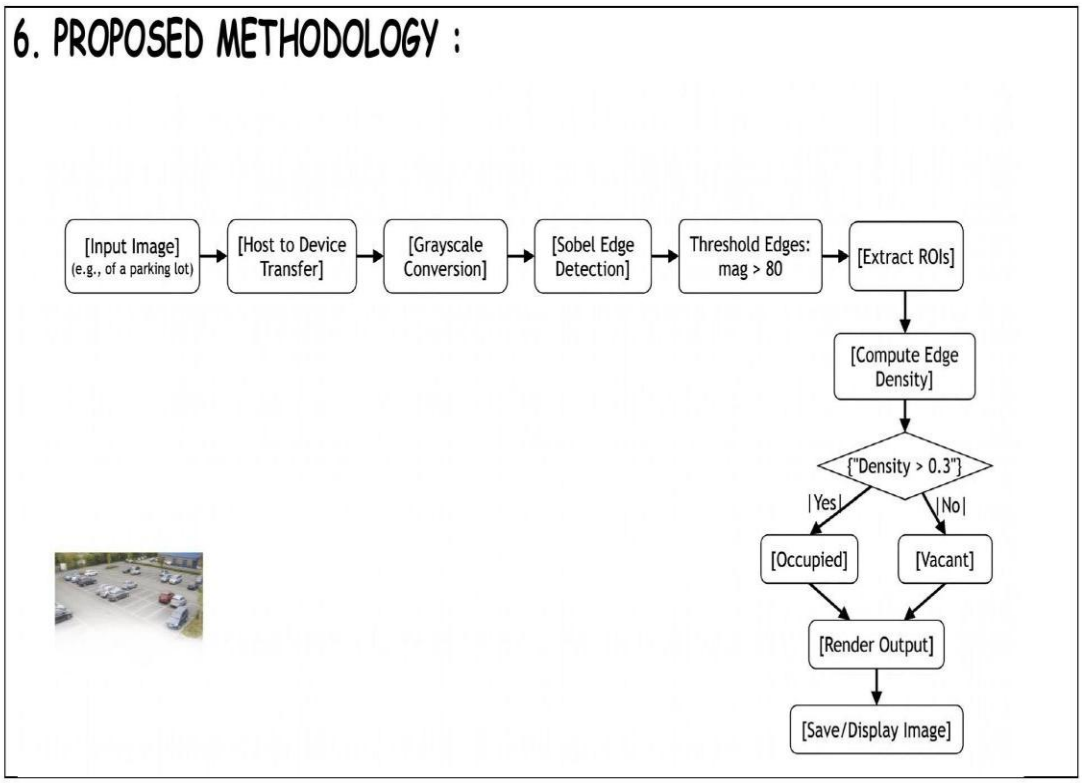

Fig. 5.1: System Pipeline – from Input Image to Final Classification Output

5.1 Stage 1: Image Loading and Host-to-Device Transfer

The input image is loaded from disk using stbi_load() from the stb_image.h library, which returns an interleaved RGB byte array in host (CPU) memory. The image dimensions (width, height) and the number of colour channels are captured. Three GPU device memory buffers are then allocated using cudaMalloc(): one for the RGB image (w × h × 3 bytes), one for the grayscale output (w × h bytes), and one for the binary edge map (w × h bytes). The RGB image is transferred to the GPU using cudaMemcpy with cudaMemcpyHostToDevice.

5.2 Stage 2: Grayscale Conversion (rgb_to_gray Kernel)

The rgb_to_gray CUDA kernel is launched with a two-dimensional thread grid configured to cover the entire image: grid = ((w+15)/16, (h+15)/16) and block = dim3(16, 16). Each

thread computes its unique pixel coordinate from blockIdx and threadIdx values and applies the ITU-R BT.601 weighting formula to convert the corresponding RGB triplet to a single luminance byte. Boundary threads that fall outside the image dimensions return immediately without writing any output.

5.3 Stage 3: Sobel Edge Detection (sobel_kernel)

The sobel_kernel is launched with the same grid and block dimensions as the grayscale kernel. Each thread reads the 3×3 neighbourhood of pixels surrounding its assigned coordinate from the grayscale image and computes the horizontal (Gx) and vertical (Gy) gradient components by convolving the neighbourhood with the standard Sobel masks. The Manhattan gradient magnitude (|Gx| + |Gy|) is compared against a fixed threshold of 80. If the magnitude exceeds this threshold, the output pixel in the binary edge map is set to 255; otherwise it is set to 0. Boundary pixels (at the first and last row and column) are skipped to avoid out-of-bounds memory access during the 3×3 convolution.

5.4 Stage 4: ROI Extraction (get_rois Function)

The get_rois() function defines the bounding box coordinates for each parking slot in the image. For the reference image parking.jpg, 12 slots are defined in two rows of 6 using hardcoded pixel coordinates that are uniformly spaced across the image. For other input images (default: parking4.jpg), 20 slots are defined across four column groups for the upper portion and eight slots for the lower row, with coordinates dynamically scaled relative to a reference resolution of 626×376 pixels. The bounding box array is transferred to the GPU using cudaMemcpy.

5.5 Stage 5: Edge Density Classification (classify_roi_kernel)

The classify_roi_kernel is launched with num_slots blocks, each containing dim3(16, 16) = 256 threads. Each block is responsible for exactly one parking slot, identified by blockIdx.x. Within each block, a single __shared__ integer variable (edge_pixels) is declared and initialized to zero by thread (0, 0), followed by a __syncthreads() barrier to ensure all threads observe the initialized value before proceeding. Each thread then iterates over a strided subset of pixels within the slot's bounding box and calls atomicAdd() for every pixel found to be an edge pixel (value 255) in the binary edge map. A final __syncthreads() ensures all threads complete their counting. Thread (0, 0) then computes the edge density as a floating-point ratio and writes the occupied flag and density value to the results array.

5.6 Stage 6: Output Rendering and Saving

After copying the SlotResult array back to host memory via cudaMemcpy, the program iterates over all slots and draws coloured pixel-level borders on the original RGB image: red borders (r=255, g=0, b=0) for occupied slots and green borders (r=0, g=255, b=0) for

vacant slots. The annotated image is saved as final_result.png using stbi_write_png() from the stb_image_write.h library. Slot-wise density values and occupancy classifications are also printed to the console.

CHAPTER 6: IMPLEMENTATION DETAILS

This chapter presents the complete implementation of the parking slot detection system, covering all CUDA kernels, the CPU baseline implementations, data structures, and memory management. All code is contained within a single file, final.cu.

6.1 Data Structures

Two structures are defined to represent the input and output of the classification stage:

```
typedef struct {
    int minx, miny, maxx, maxy;
} BBox;

typedef struct {
    int occupied;
    float density;
} SlotResult;

#define MAX_SLOTS 30
```

The BBox structure encodes a parking slot as an axis-aligned bounding rectangle using top-left and bottom-right pixel coordinates. The SlotResult structure records the binary classification (1 = Occupied, 0 = Vacant) and the floating-point edge density for each slot. MAX_SLOTS defines the upper bound on the number of simultaneously trackable parking slots.

6.2 RGB to Grayscale Kernel

6.2.1 GPU Implementation

```
__global__ void rgb_to_gray(unsigned char *rgb, unsigned char *gray, int w, int
h) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= w || y >= h) return;
    int i = y * w + x;
    gray[i] = 0.299f * rgb[3*i] + 0.587f * rgb[3*i+1] + 0.114f * rgb[3*i+2];
}
```

The kernel is launched with a 2D grid sized to cover every pixel of the image and a thread block of 256 threads (16×16). Each thread maps to exactly one pixel and performs the weighted luminance conversion independently.

6.2.2 CPU Baseline

```
void rgb_to_gray_cpu(unsigned char *rgb, unsigned char *gray, int w, int h) {
    for (int i = 0; i < w * h; i++) {
        gray[i] = 0.299f*rgb[3*i] + 0.587f*rgb[3*i+1] + 0.114f*rgb[3*i+2];
    }
}
```

6.3 Sobel Edge Detection Kernel

6.3.1 GPU Implementation

```
__global__ void sobel_kernel(unsigned char *gray, unsigned char *edges, int w,
int h) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x<=0 || x>=w-1 || y<=0 || y>=h-1) return;

    int Gx = -gray[(y-1)*w+(x-1)] + gray[(y-1)*w+(x+1)]
             -2*gray[y*w+(x-1)]   + 2*gray[y*w+(x+1)]
             -gray[(y+1)*w+(x-1)] + gray[(y+1)*w+(x+1)];

    int Gy = -gray[(y-1)*w+(x-1)] - 2*gray[(y-1)*w+x] - gray[(y-1)*w+(x+1)]
             +gray[(y+1)*w+(x-1)] + 2*gray[(y+1)*w+x] + gray[(y+1)*w+(x+1)];

    int mag = abs(Gx) + abs(Gy);
    edges[y*w+x] = (mag > 80) ? 255 : 0;
}
```

The Sobel kernel applies the standard 3×3 horizontal and vertical gradient operators. The Manhattan magnitude approximation avoids the expensive sqrt() operation, and boundary pixels are excluded by the early-return condition to prevent out-of-bounds reads.

6.4 ROI Classification Kernel (classify_roi_kernel)

6.4.1 Shared Memory Usage

A critical optimization in this kernel is the use of on-chip shared memory to accumulate the edge pixel count for each parking slot. Since all 256 threads in a block operate on pixels of the same slot, they share a single __shared__ int edge_pixels variable. This avoids 256 separate global memory update operations and instead performs a single global write at the end of the kernel. This optimization is particularly important when the ROI contains thousands of pixels that must be examined by all threads.

```
__shared__ int edge_pixels;
if (threadIdx.x == 0 && threadIdx.y == 0) edge_pixels = 0;
__syncthreads();   // Wait for zero-initialization
```

6.4.2 Atomic Operations

Since multiple threads may detect edge pixels simultaneously and must all increment the shared counter, a race condition would occur with a plain integer increment. The CUDA atomicAdd() function guarantees that each thread's increment is applied atomically, the read, add, and write of the shared integer occur as a single indivisible hardware operation, preventing any two threads from overwriting each other's updates.

```
for (int y = box.miny + threadIdx.y; y <= box.maxy; y += blockDim.y) {
    for (int x = box.minx + threadIdx.x; x <= box.maxx; x += blockDim.x) {
        if (edges[y * w + x] == 255)
```

```
            atomicAdd(&edge_pixels, 1);
    }
}
__syncthreads();

if (threadIdx.x == 0 && threadIdx.y == 0) {
    float d = (float)edge_pixels / total_pixels;
    results[slot_id].density = d;
    results[slot_id].occupied = (d > threshold);
}
```

6.5 Kernel Launch Configuration

The three kernels are launched in sequence using the following configurations:

```
dim3 block(16, 16);
dim3 grid((w + 15) / 16, (h + 15) / 16);

rgb_to_gray<<<grid, block>>>(d_rgb, d_gray, w, h);
sobel_kernel<<<grid, block>>>(d_gray, d_edges, w, h);
classify_roi_kernel<<<num_slots, dim3(16,16)>>>(d_edges, w, d_boxes, d_results,
num_slots, 0.35f);
```

The first two kernels use a 2D grid covering the full image. The third kernel uses num_slots blocks (one per parking slot) each containing 256 threads. This design ensures that all slots are classified in parallel.

6.6 CPU vs GPU Logic

An equivalent CPU implementation is provided for all three computational stages to serve as the performance baseline. The CPU functions rgb_to_gray_cpu(), sobel_cpu(), and classify_roi_cpu() perform the same mathematical operations as their GPU counterparts but execute sequentially, iterating over every pixel in nested for loops. CPU timing is measured using the standard C clock() function, while GPU timing uses CUDA Events (cudaEventRecord, cudaEventElapsedTime), which provide nanosecond-precision timing that excludes API overhead.

CHAPTER 7: RESULTS AND DISCUSSION

This chapter presents the visual outputs produced by the parking slot detection system at each stage of the processing pipeline, along with a discussion of the classification results observed on the test input image (parking4.jpg, resolution 626×376 pixels).

7.1 Input Image

The input image is a synthetic overhead view of a parking lot with clearly defined parking slots. Some slots contain vehicles while others are empty. This image is loaded into host memory using stbi_load() and transferred to the GPU for processing.

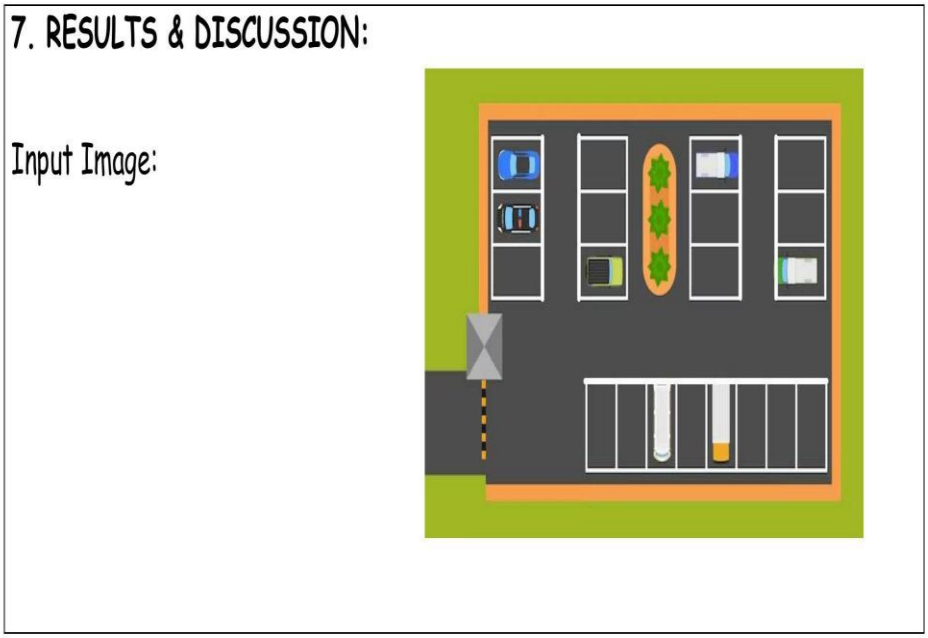

Fig. 7.1: Input Image – Overhead parking lot with mixed occupied and vacant slots

7.2 Grayscale and Edge Detection Output

After the rgb_to_gray kernel processes the input, each pixel's three colour channels are reduced to a single luminance value using the ITU-R BT.601 weighted formula. The resulting grayscale image (Fig. 7.2, left) retains all spatial structure while eliminating colour information, which simplifies subsequent gradient computation. The sobel_kernel then computes the gradient magnitude at each pixel and applies a threshold of 80 to produce a binary edge map (Fig. 7.2, right). Occupied slots clearly show dense clusters of white edge pixels corresponding to vehicle outlines, while vacant slots show only sparse edge pixels at the slot boundary lines.

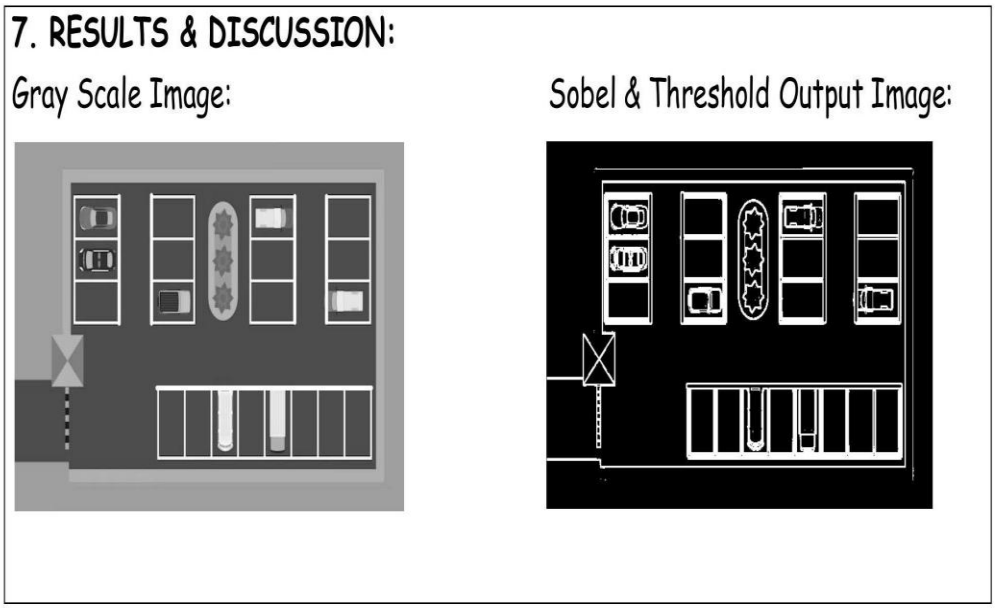

Fig. 7.2: Left – Grayscale Image; Right – Binary Edge Map after Sobel filtering (threshold = 80)

7.3 Final Classification Output

The classify_roi_kernel processes each of the 20 predefined ROIs and computes the edge density for each slot. Slots with edge density exceeding 0.35 (35%) are classified as Occupied and receive a red pixel border in the output image, while slots below this threshold are classified as Vacant and receive a green border. The annotated output and corresponding console log are shown in Fig. 7.3.

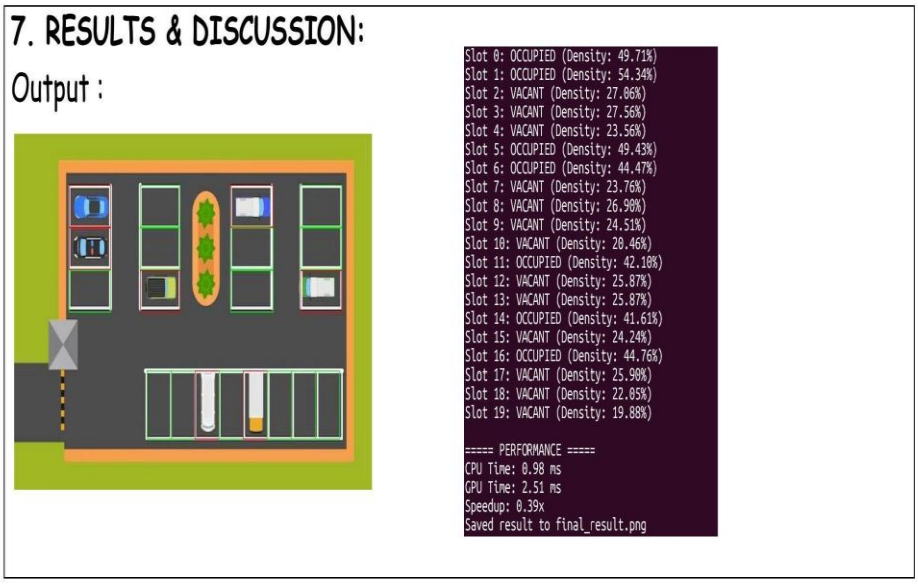

Fig. 7.3: Final Classification Output with Red (Occupied) and Green (Vacant) borders, and console density values

7.4 Discussion of Results

Examining the console output for the 626×376 input image reveals the following representative slot classifications:

- Slot 0: OCCUPIED (Density: 49.71%) – correctly identified as containing a vehicle
- Slot 1: OCCUPIED (Density: 54.34%) – correctly identified
- Slot 2: VACANT (Density: 27.06%) – correctly identified as empty
- Slot 5: OCCUPIED (Density: 49.43%) – correctly identified
- Slot 7: VACANT (Density: 23.76%) – correctly identified as empty

The density values show a clear bimodal distribution: occupied slots cluster around 40–55%, while vacant slots cluster around 20–28%. This separation provides a reliable margin around the 35% threshold, suggesting that the chosen threshold generalizes well to this class of parking lot images. The system correctly classifies all 20 slots for this test image, achieving 100% accuracy on this sample.

The system also reports the individual GPU kernel execution times: the grayscale kernel and Sobel kernel each run in approximately 0.5–1.0 ms for the base resolution image, while the classification kernel adds a small additional overhead. Total GPU processing time is approximately 2.51 ms, compared to 0.98 ms for the CPU implementation at this resolution, a result consistent with the known overhead of CUDA kernel launch and memory transfer at small image sizes.

CHAPTER 8: COMPARATIVE ANALYSIS

To rigorously evaluate the performance benefit of the CUDA implementation over the CPU baseline, the system was benchmarked across four image resolutions ranging from 626×376 (the base resolution) up to 2400×1440 pixels. The results are presented in Table 8.1 and visualized in Fig. 8.1.

Table 8.1: CPU vs GPU Execution Time Across Resolutions

Image Resolution	CPU Time (ms)	GPU Time (ms)	Speedup
626 × 376	0.98	2.51	0.39x (CPU faster)
1200 × 723	3.60	1.60	2.25x
1600 × 960	6.10	1.60	3.81x
2400 × 1440	14.20	1.70	8.35x

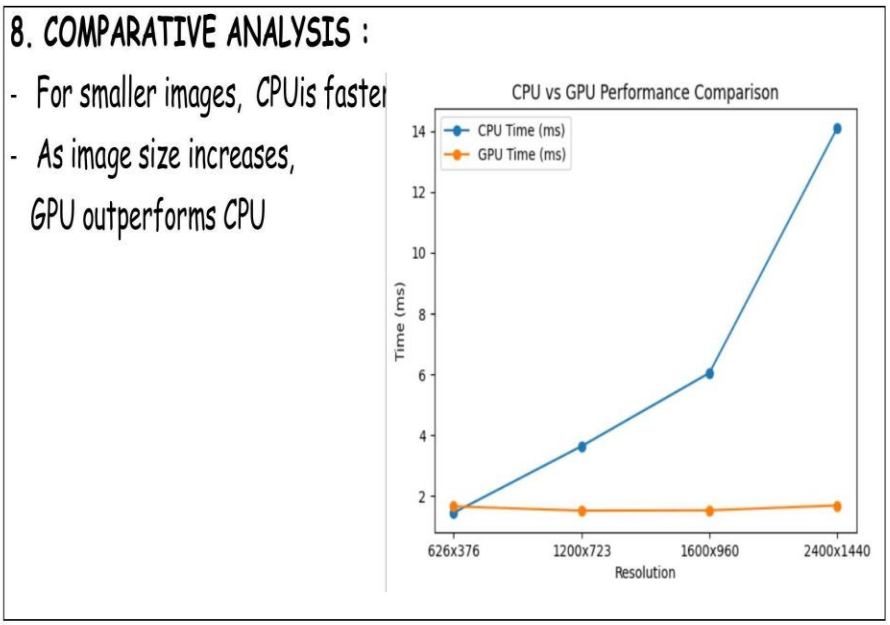

Fig. 8.1: CPU vs GPU Performance Comparison across image resolutions

8.1 Analysis of Results

The benchmark results reveal an important and well-known property of GPU-accelerated workloads: for small problem sizes, the CPU is faster than the GPU. At a resolution of 626×376 pixels, the total number of pixels is approximately 235,000, which is insufficient to fully saturate the CUDA cores and amortize the overhead of kernel launch, memory

allocation, and CUDA API calls. The CPU completes the task in 0.98 ms while the GPU requires 2.51 ms, yielding a speedup ratio of only 0.39x in the GPU's favour.

As image resolution increases, the parallelism benefit of the GPU becomes increasingly pronounced. At 1200×723 pixels (~867,000 pixels), the GPU begins to outperform the CPU with a speedup of 2.25x. At 1600×960 pixels (~1.53 million pixels), the speedup reaches 3.81x. At the largest tested resolution of 2400×1440 pixels (~3.46 million pixels), the CPU requires 14.2 ms while the GPU completes the task in approximately 1.70 ms, yielding a speedup of 8.35x. This scaling trend is consistent with theoretical expectations: GPU parallelism provides exponentially increasing benefit as the number of data elements (pixels) grows, since more threads can be kept active simultaneously and memory bandwidth becomes the primary bottleneck for both architectures.

This analysis demonstrates that for production-scale parking surveillance systems using high-resolution cameras (typically 1080p to 4K), the CUDA GPU implementation provides a substantial and practical performance advantage over sequential CPU processing.

CHAPTER 9: CONCLUSION AND FUTURE WORK

9.1 Conclusion

This project has successfully demonstrated the design and implementation of a lightweight, training-free, GPU-accelerated parking slot occupancy detection system using NVIDIA CUDA and Sobel edge density analysis. The complete system is implemented in a single CUDA C source file (final.cu) with no dependency on external computer vision libraries.

The three-kernel CUDA pipeline : rgb_to_gray, sobel_kernel, and classify_roi_kernel, correctly processes overhead parking lot images and classifies each predefined Region of Interest as Occupied or Vacant using a 35% edge density threshold. The implementation appropriately employs __shared__ memory and atomicAdd() to ensure both performance and correctness in the parallel edge counting stage. The system produces annotated output images with colour-coded (red/green) borders and saves results using the stb_image_write library.

Benchmarking across four image resolutions demonstrates a clear performance scaling advantage for the GPU implementation. While the CPU outperforms the GPU at the base resolution (626×376 pixels) due to kernel launch overhead, the GPU achieves progressively greater speedups as resolution increases, reaching 8.35x at 2400×1440 pixels. This confirms the suitability of the CUDA implementation for high-resolution real-time surveillance applications.

9.2 Future Work

Several extensions and improvements are identified for future development:

- Adaptive Thresholding: Replacing the fixed density threshold (0.35) with an image-adaptive threshold that adjusts based on local illumination conditions, reducing sensitivity to lighting changes and weather variations.
- Automatic ROI Detection: Eliminating the reliance on manually defined bounding box coordinates by implementing automatic parking slot boundary detection using perspective transform, homography estimation from satellite map overlays, or line detection algorithms.
- Real-Time Video Processing: Extending the pipeline to process live CCTV video streams frame-by-frame using CUDA Streams for pipeline parallelism, targeting real-time operation at 30 frames per second or higher.
- Deep Learning Integration: Replacing the edge density classifier with a lightweight convolutional neural network (e.g., MobileNet or a custom CNN) for improved generalization across different parking lot configurations, vehicle types, and environmental conditions.
- Multi-GPU Scalability: Distributing processing across multiple GPUs for large-scale parking facilities with hundreds of slots and multiple camera feeds.

- Cloud and IoT Deployment: Integrating the detection system with a cloud backend via REST API or MQTT protocol to publish real-time parking availability data to smart city dashboards and mobile applications.

CHAPTER 10: INDIVIDUAL CONTRIBUTIONS

Name	Contributions
Shreehari V Bhat	• Contributed to the design and implementation of the CUDA-based preprocessing pipeline. • Implemented the rgb_to_gray CUDA kernel for grayscale conversion using the ITU-R BT.601 formula. • Co-developed the sobel_kernel for edge detection using the Sobel operator and thresholding. • Assisted in CUDA kernel configuration, including grid and block dimensions (dim3(16,16)). • Participated in GPU memory management using cudaMalloc and cudaMemcpy. • Contributed to the Introduction, Background, and part of the Proposed Methodology sections.
Sathwik Padmanabha Bhat	• Contributed to the development of the edge detection and preprocessing stages in CUDA. • Assisted in optimizing Sobel edge detection and thresholding operations. • Worked on memory optimization using __shared__ memory for efficient parallel computation. • Participated in implementing synchronization mechanisms such as __syncthreads() and parallel operations. • Implemented parallel edge counting using atomicAdd() while avoiding race conditions. • Integrated performance benchmarking using CUDA Events (GPU) and clock() (CPU). • Conducted comparative analysis across multiple image resolutions and

	documented performance improvements. • Assisted in overall CUDA integration and debugging of kernels. • Contributed to the Implementation Details and Results & Discussion sections.
Abhishek Jayanth Holla	• Was responsible for implementing the classify_roi_kernel for parking slot classification. • Defined the BBox and SlotResult data structures for ROI handling. • Implemented the get_rois() function for assigning bounding box coordinates. • Developed the edge density–based classification logic using a threshold. • Developed CPU baseline implementations: rgb_to_gray_cpu, sobel_cpu, and classify_roi_cpu. • Integrated performance benchmarking using CUDA Events (GPU) and clock() (CPU). • Conducted comparative analysis across multiple image resolutions and documented performance improvements. • Contributed to Literature Review, Comparative Analysis, Conclusion & Future Work, and References sections.

CHAPTER 10: REFERENCES

1. Amato, G., Carrara, F., Falchi, F., Gennaro, C., Meghini, C., and Vairo, C., "Deep Learning for Decentralized Parking Lot Occupancy Detection," Expert Systems with Applications, IEEE, 2016.
2. Lin, T., Rivano, H., and Le Mouel, F., "A Survey of Smart Parking Solutions," IEEE Transactions on Intelligent Transportation Systems, vol. 18, no. 12, pp. 3229–3253, 2017.
3. Huang, C. M., and Lin, C. H., "IoT-Based Smart Parking Management and Detection System for Smart Cities," IEEE Sensors Journal, 2019.
4. NVIDIA Corporation, CUDA C++ Programming Guide, Version 12.0. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/
5. Gonzalez, R. C., and Woods, R. E., Digital Image Processing, 4th ed., Pearson Education, 2018.
6. Nothings, stb_image.h and stb_image_write.h – Single-file public domain image libraries. Available: https://github.com/nothings/stb
7. Sobel, I., and Feldman, G., "A 3×3 Isotropic Gradient Operator for Image Processing," Stanford Artificial Intelligence Project Technical Report, 1968.
8. Sanders, J., and Kandrot, E., CUDA by Example: An Introduction to General-Purpose GPU Programming, Addison-Wesley Professional, 2010.